DSA Assignment #A: 5月份月考

2500010774 兰玉琪 数学科学学院

1. 题目

E04137: 最小新整数

monotonic stack, http://cs101.openjudge.cn/practice/04137/

思路：

代码：

```
for _ in range(int(input())):
    n,k = map(int,input().split())
    n = str(n)
    stack = []
    t = 0
    for i in range(len(n)):
        num = n[i]
        while stack and t < k:
            if stack[-1] > num:
                stack.pop()
                t += 1
            else:
                break
        stack.append(num)
    if t == k:
        print(int("".join(stack)))
    else:
        print(int("".join(stack[:-(k-t)])))
```

#52475984提交状态

View Submit Statistics Clarify

状态: Accepted

Source Code

```
for _ in range(int(input())):
    n,k = map(int,input().split())
    n = str(n)
    stack = []
    t = 0
    for i in range(len(n)):
        num = n[i]
        while stack and t < k:
            if stack[-1] > num:
                stack.pop()
                t += 1
            else:
                break
        stack.append(num)
    if t == k:
        print(int("".join(stack)))
    else:
        print(int("".join(stack[:-(k-t)])))
```

基本信息

#: 52475984
题目: E04137
提交人: 25n2500010774(sleepy(25n2500010774)
内存: 3512kB
时间: 20ms
语言: Python3
提交时间: 2026-05-06 15:13:38

©2002-2022 POJ 京ICP备20010980号-1

中文 Help About

E04143: 和为给定数

two pointers, http://cs101.openjudge.cn/dsapre/04143/

思路:

代码:

```
n = int(input())
nums = sorted(list(map(int,input().split())))
m = int(input())
visited = set()
x,y = -1,-1
for num in nums:
    if m - num in visited:
        x,y = num,m-num
    visited.add(num)
if x == -1 and y == -1:
    print("No")
else:
    print(min(x,y),max(x,y))
```

#52476083提交状态

View Submit Statistics Clarify

状态: Accepted

Source Code

```
n = int(input())
nums = sorted(list(map(int,input().split())))
m = int(input())
visited = set()
x,y = -1,-1
for num in nums:
    if m - num in visited:
        x,y = num,m-num
    visited.add(num)
if x == -1 and y == -1:
    print("No")
else:
    print(min(x,y),max(x,y))
```

基本信息

#: 52476083
题目: E04143
提交人: 25n2500010774(sleepy(25n2500010774)
内存: 16040kB
时间: 73ms
语言: Python3
提交时间: 2026-05-06 15:18:13

©2002-2022 POJ 京ICP备20010980号-1

M27638: 求二叉树的高度和叶子数目

http://cs101.openjudge.cn/practice/27638/

思路:

代码:

```
n = int(input())
class TreeNode:
    def __init__(self,val):
        self.val = val
        self.left = None
        self.right = None
nodes = [TreeNode(i) for i in range(n)]
is_leaf = [False for i in range(n)]
is_root = [True for i in range(n)]
for i in range(n):
    node = nodes[i]
    t,s = map(int,input().split())
    if t != -1:
        node.left = nodes[t]
        is_root[t] = False
    if s != -1:
        node.right = nodes[s]
        is_root[s] = False
def height(node):
    if not node:
        return -1
    if not node.left and not node.right:
        is_leaf[node.val] = True
    return max(height(node.left),height(node.right))+1
print(height(nodes[is_root.index(True)]),is_leaf.count(True))
```

#52476284提交状态

View Submit Statistics Clarify

状态: Accepted

基本信息

#: 52476284
题目: M27638
提交人: 25n2500010774(sleepy(25n2500010774)
内存: 3652kB
时间: 19ms
语言: Python3
提交时间: 2026-05-06 15:26:46

Source Code

```
n = int(input())
class TreeNode:
    def __init__(self,val):
        self.val = val
        self.left = None
        self.right = None
nodes = [TreeNode(i) for i in range(n)]
is_leaf = [False for i in range(n)]
is_root = [True for i in range(n)]
for i in range(n):
    node = nodes[i]
    t,s = map(int,input().split())
    if t != -1:
        node.left = nodes[t]
        is_root[t] = False
    if s != -1:
        node.right = nodes[s]
        is_root[s] = False
def height(node):
    if not node:
        return -1
    if not node.left and not node.right:
        is_leaf[node.val] = True
    return max(height(node.left),height(node.right))+1
print(height(nodes[is_root.index(True)]),is_leaf.count(True))
```

M30720: 败方树的构建与维护

http://cs101.openjudge.cn/practice/30720/

思路:

代码:

```
from collections import deque
import math
class TreeNode:
    def __init__(self,val,i):
        self.index = i
        self.val = val
        self.left = None
        self.right = None
        self.parent = None
n,m = map(int,input().split())
t = math.ceil(math.log2(n))
mylist = list(map(int,input().split()))+[float("inf") for _ in range(2**t - n)]
tree = [TreeNode(mylist[i],i) for i in range(len(mylist))]
nodes = deque([node for node in tree])
while len(nodes) >= 2:
    node1 = nodes.popleft()
    node2 = nodes.popleft()
    if node1.val < node2.val:
        node = TreeNode(node1.val,node1.index)
    else:
        node = TreeNode(node2.val,node2.index)
    node.left = node1
    node.right = node2
    node1.parent = node
    node2.parent = node
    nodes.append(node)
root = nodes[0]
visited = [False for _ in range(len(mylist))]
visited[root.index] = True
q = deque([root])
res = [root.val]
while q:
    for _ in range(len(q)):
        node = q.popleft()
        if node.left:
            if not visited[node.left.index]:
                if node.left.val != float("inf"):
                    res.append(node.left.val)
                visited[node.left.index] = True
            q.append(node.left)
        if node.right:
            if not visited[node.right.index]:
                if node.right.val != float("inf"):
```

```
                    res.append(node.right.val)
                visited[node.right.index] = True
            q.append(node.right)
print(*res)
for _ in range(m):
    i,num = map(int,input().split())
    tree[i].val = num
    node = tree[i]
    while node.parent:
        if node.parent.left == node:
            if node.val < node.parent.right.val:
                node.parent.val = node.val
                node.parent.index = node.index
                node = node.parent
            else:
                if node.parent.val == node.parent.right.val:
                    break
                else:
                    node.parent.val = node.parent.right.val
                    node.parent.index = node.parent.right.index
                    node = node.parent
        else:
            if node.val < node.parent.left.val:
                node.parent.val = node.val
                node.parent.index = node.index
                node = node.parent
            else:
                if node.parent.val == node.parent.left.val:
                    break
                else:
                    node.parent.val = node.parent.left.val
                    node.parent.index = node.parent.left.index
                    node = node.parent
    visited = [False for _ in range(len(mylist))]
    visited[root.index] = True
    q = deque([root])
    res = [root.val]
    while q:
        for _ in range(len(q)):
            node = q.popleft()
            if node.left:
                if not visited[node.left.index]:
                    if node.left.val != float("inf"):
```

```
                    res.append(node.left.val)
                visited[node.left.index] = True
            q.append(node.left)
        if node.right:
            if not visited[node.right.index]:
                if node.right.val != float("inf"):
                    res.append(node.right.val)
                visited[node.right.index] = True
            q.append(node.right)
print(*res)
```

#52478311提交状态

View Submit Statistics Clarify

状态: Accepted

Source Code

```
from collections import deque
import math
class TreeNode:
    def __init__(self,val,i):
        self.index = i
        self.val = val
        self.left = None
        self.right = None
        self.parent = None
n,m = map(int,input().split())
t = math.ceil(math.log2(n))
mylist = list(map(int,input().split()))+[float("inf") for _ in range(2*
tree = [TreeNode(mylist[i],i) for i in range(len(mylist))]
nodes = deque([node for node in tree])
while len(nodes) >= 2:
    node1 = nodes.popleft()
    node2 = nodes.popleft()
    if node1.val < node2.val:
        node = TreeNode(node1.val,node1.index)
    else:
        node = TreeNode(node2.val,node2.index)
    node.left = node1
    node.right = node2
    node1.parent = node
    node2.parent = node
    nodes.append(node)
```

基本信息

#: 52478311
题目: 30720
提交人: 25n2500010774(sleepy(25n2500010774)
内存: 7204kB
时间: 809ms
语言: Python3
提交时间: 2026-05-06 17:09:46

27093: 排队又来了

Segment Tree, Discretization（离散化）, binary search, http://cs101.openjudge.cn/practice/27093/

思路:

复健一下树状数组

学的是题解中的这个方法

代码

```python
import sys
import bisect

# 增加递归深度限制（虽然本解法主要通过迭代实现，这在大规模数据下是好习惯）
sys.setrecursionlimit(200000)

def solve():
    # 使用 sys.stdin.read 一次性读取所有输入，显著提升 Python I/O 速度
    input_data = sys.stdin.read().split()

    if not input_data:
        return

    iterator = iter(input_data)

    try:
        N = int(next(iterator))
        D = int(next(iterator))
        # 读取 N 个同学的身高
        h = [int(next(iterator)) for _ in range(N)]
    except StopIteration:
        return

    # --- 1. 离散化 (Coordinate Compression) ---
    # 身高数值范围很大 (1e9)，不能直接作为数组下标。
    # 但 N 只有 1e5，所以将其映射到 [0, M-1] 的排名 (Rank)。
    vals = sorted(list(set(h)))
    rank_map = {v: i for i, v in enumerate(vals)}
    M = len(vals)

    # --- 2. 定义树状数组 (Fenwick Tree) ---
    # bitL: 用于查询值域 [0, val - D - 1] 内的最大层数
    # bitR: 用于查询值域 [val + D + 1, max_val] 内的最大层数
    # 数组大小设为 M + 1 (因为树状数组通常使用 1-based 索引)
    bitL = [0] * (M + 1)
    bitR = [0] * (M + 1)

    # 单点更新：将 idx 位置的值更新为 val (取最大值)
    def bit_update(bit, idx, val):
        limit = len(bit)
        while idx < limit:
            if val > bit[idx]:
                bit[idx] = val
```

```
        idx += idx & (-idx)

# 前缀查询：查询 [1, idx] 范围内的最大值
def bit_query(bit, idx):
    res = 0
    while idx > 0:
        if bit[idx] > res:
            res = bit[idx]
        idx -= idx & (-idx)
    return res

# --- 3. 计算每个同学的层数 ---
# layer_buckets 用于收集每一层的同学身高
# 字典结构：{层数: [身高1, 身高2, ...]}
layer_buckets = {}
max_layer_global = 0

for val in h:
    r = rank_map[val] # 获取当前身高的离散化排名 (0 ~ M-1)

    # --- 计算左侧阻塞的最大层数 ---

    # 情况 A: 被身高太小的人阻塞 (h_k < val - D)
    # bisect_left 找到第一个 >= val-D 的位置，其左侧即为 < val-D
    idx_small = bisect.bisect_left(vals, val - D)
    # 查询 bitL 在范围 [0, idx_small-1] 的最大层数 (对应BIT下标 idx_small)
    max_L = bit_query(bitL, idx_small)

    # 情况 B: 被身高太大的人阻塞 (h_k > val + D)
    # bisect_right 找到第一个 > val+D 的位置，该位置及右侧即为 > val+D
    idx_large = bisect.bisect_right(vals, val + D)

    # 在 bitR 中，我们使用反转映射技巧：
    # 实际排名 rank 越大，映射到 bitR 的下标 M - rank 越小。
    # 这样查询 bitR 的前缀最大值，实际上就是查询原数组的后缀最大值。
    # 范围 [idx_large, M-1] 映射到 bitR 下标 [1, M - idx_large]
    max_R = 0
    if idx_large < M:
        pos_in_bitR = M - idx_large
        max_R = bit_query(bitR, pos_in_bitR)

    # 当前同学的层数 = 所有阻塞者中的最大层数 + 1
    cur_layer = max(max_L, max_R) + 1
```

```
            # 记录全局最大层数以便后续输出
            if cur_layer > max_layer_global:
                max_layer_global = cur_layer

            # 将身高放入对应的层桶中
            if cur_layer not in layer_buckets:
                layer_buckets[cur_layer] = []
            layer_buckets[cur_layer].append(val)

            # --- 更新树状数组 ---
            # 将当前同学的信息加入 BIT，成为后续同学可能的阻塞者

            # 更新 bitL: 正常映射，下标为 r + 1
            bit_update(bitL, r + 1, cur_layer)

            # 更新 bitR: 反转映射，下标为 M - r
            bit_update(bitR, M - r, cur_layer)

    # --- 4. 输出结果 ---
    # 字典序最小原则：
    # 1. 必须按层数从小到大输出 (拓扑序)
    # 2. 同一层内互不阻塞，按身高从小到大输出 (贪心)

    output = []
    for layer in range(1, max_layer_global + 1):
        if layer in layer_buckets:
            # 对该层的所有身高进行排序
            members = sorted(layer_buckets[layer])
            for v in members:
                output.append(str(v))

    sys.stdout.write('\n'.join(output) + '\n')

if __name__ == '__main__':
    solve()
```

这题本来就不会，放一个用别人代码ac的图也没多大意义，故而没有放

T30669: 地铁换乘

LCA, binary lifting, http://cs101.openjudge.cn/practice/30669/

思路：

代码

```python
from collections import defaultdict
n,t = map(int,input().split())
t -= 1
linjiebiao = defaultdict(set)
for _ in range(n-1):
    u,v = map(int,input().split())
    u -= 1
    v -= 1
    linjiebiao[u].add(v)
    linjiebiao[v].add(u)
parent = [-1 for _ in range(n)]
visited = [False for _ in range(n)]
visited[t] = True
stack = [t]
while stack:
    num = stack.pop()
    for num1 in linjiebiao[num]:
        if not visited[num1]:
            parent[num1] = num
            visited[num1] = True
            stack.append(num1)
p,q,v1,v2 = map(int,input().split())
p -= 1
q -= 1
cur = q
q_rout = [q]
while parent[cur] != -1:
    cur = parent[cur]
    q_rout.append(cur)
myset = set(q_rout)
if p == q:
    print((len(myset)-1)//(v1+v2),(len(myset))-1)
else:
    cur = p
    p_rout = [p]
    day = 0
    flag = False
    depth = 0
    while parent[cur] != -1:
        cur = parent[cur]
        p_rout.append(cur)
        if cur in myset:
            flag = True
```

```
        if flag:
            depth += 1
    day = (len(myset)+len(p_rout)-2*depth)//(v1+v2)
    if day*v2-(len(q_rout)-depth) >= 0:
        depth = day*v2-(len(q_rout)-depth)+depth-1
    else:
        depth = (len(q_rout)-depth)-day*v2+depth-1
    print(day,depth)
```

#52478455提交状态

View Submit Statistics Clarify

状态: Accepted

Source Code

```
from collections import defaultdict
n,t = map(int,input().split())
t -= 1
linjiebiao = defaultdict(set)
for _ in range(n-1):
    u,v = map(int,input().split())
    u -= 1
    v -= 1
    linjiebiao[u].add(v)
    linjiebiao[v].add(u)
parent = [-1 for _ in range(n)]
visited = [False for _ in range(n)]
visited[t] = True
stack = [t]
while stack:
    num = stack.pop()
    for num1 in linjiebiao[num]:
        if not visited[num1]:
            parent[num1] = num
            visited[num1] = True
            stack.append(num1)
p,q,v1,v2 = map(int,input().split())
```

基本信息

#: 52478455
题目: 30669
提交人: 25n2500010774(sleepy(25n2500010774)
内存: 81208kB
时间: 1208ms
语言: Python3
提交时间: 2026-05-06 17:18:31

2. 学习总结和个人收获

遗憾止步ac4，我在考场上4的代码都搓出来了，但是没啥时间来看6了，太可惜了，否则就可以ac5了。第5题一看就是我不会做的题。